Efficient Computation of Closed Substrings

Samkith K. Jain and Neerja Mhaskar(✉)

Department of Computing and Software, McMaster University, Hamilton, Canada
{kishors,pophlin}@mcmaster.ca

Abstract. A ***closed string*** u is either of length one or contains a border that occurs only as a prefix and as a suffix in u and nowhere else within u. In this paper, we present a fast $\mathcal{O}(n \log n)$ time algorithm to compute all $\mathcal{O}(n^2)$ closed substrings by introducing a compact representation for all closed substrings of a string $w[1..n]$, using only $\mathcal{O}(n \log n)$ space. We also present a simple and space-efficient solution to compute all maximal closed substrings (MCSs) using the suffix array (SA) and the longest common prefix (LCP) array of $w[1..n]$. Finally, we show that the exact number of MCSs ($M(f_n)$) in a Fibonacci word f_n, for $n \geq 5$, is $\approx \left(1 + \frac{1}{\phi^2}\right) F_n \approx 1.382 F_n$, where ϕ is the golden ratio.

Keywords: Closed Strings · Maximal Closed Substrings · Fibonacci Words

1 Introduction

A ***closed string*** u is either of length one or contains a border that occurs only as a prefix and as a suffix in u and nowhere else within u; otherwise, it is called ***open***. In other words, the longest (possibly overlapping) border of a closed substring u does not have internal occurrences in u. For example, in the string $abcab$, ab occurs only as a prefix and a suffix. An occurrence of a substring u of a string w is said to be ***maximal right (left)-closed*** if it is a closed string and cannot be extended to the right (left) by a character to form another closed string. A ***maximal closed substring (MCS)*** of a string w is a substring that is both maximal left and maximal right-closed. An MCS of length one is called a ***singleton*** MCS; otherwise, it is called a ***non-singleton*** MCS. For example, in string $w[1..8] = abaccaba$, the non-singleton MCSs are $w[1..8] = abaccaba$, $w[3..6] = acca$, $w[1..3] = w[6..8] = aba$, $w[4..5] = cc$, and the singleton MCSs are all occurrences of a and b. Observe that $w[2..7] = baccab$, $w[2..8] = baccaba$, $w[1..7] = abaccab$, and both occurrences of c are closed but not maximal.

N. Mhaskar—Research funded by the Natural Sciences & Engineering Research Council of Canada [Grant Number RGPIN-2024-06915].

G. Badkobeh et al. (Eds.): SPIRE 2025, LNCS 16073, pp. 172–187, 2026.
https://doi.org/10.1007/978-3-032-05228-5_15

The concept of open and closed strings was introduced by Fici in [12] to study the combinatorial properties of strings, with connections to Sturmian and Trapezoidal words. Then concepts of *Longest Closed Factorization* and *Longest Closed Factor Array* were introduced in [2], with improved solutions later proposed in [7]. Later, [3] introduced efficient algorithms for minimal and shortest closed factors. Related problems such as the k-closed string problem [1] and combinatorial problems on closed factors in Arnoux-Rauzy and m-bonacci words [14,24] have also been explored.

It has been shown that a string contains at most $\mathcal{O}(n^2)$ *distinct* closed factors [5]. More recently (in 2024), [23] showed that the maximal number of distinct closed factors in a word of length n is $\frac{n^2}{6}$. In early 2025, [20] presented a linear algorithm to count the number of distinct closed factors of a string in $\mathcal{O}(n \log \sigma)$ time and another in $\mathcal{O}(n)$ time. They also described methods to enumerate all closed and distinct closed factors, achieving a complexity of $\mathcal{O}(n^2)$, which is further optimized to $\mathcal{O}(n\sqrt{\log n} + \text{output} \cdot \log n)$ for distinct closed substrings, using weighted ancestor queries.

In [4] Badkobeh et al. introduced the notion of MCSs and proposed a $\mathcal{O}(n\sqrt{n})$ loose upper bound on the number of MCSs in a string of length n. They also proposed an algorithm using suffix trees to compute the MCSs for strings on a binary alphabet in $\mathcal{O}(n \log n + n\sqrt{n})$ time. Badkobeh et al. in [6] extended their algorithm to a general alphabet with a time complexity of $\mathcal{O}(n \log n)$, implicitly improving the bound on the total number of MCSs, this was later directly proved by Kosolobov in [18].

In this paper, we propose solutions to the following problems:

Compact Representation of All $\mathcal{O}(n^2)$ Closed Substrings: Given a string $w[1..n]$ we define a compact representation of all $\mathcal{O}(n^2)$ closed substrings of w using $\mathcal{O}(n \log n)$ space.

Compute All $\mathcal{O}(n^2)$ Closed Substrings: Given a string $w[1..n]$, we compute all $\mathcal{O}(n^2)$ closed substrings of w in a compact representation using the suffix array (SA) and the longest common prefix array (LCP), in $\mathcal{O}(n \log n)$ time.

Compute All MCSs: Given a string $w[1..n]$ we find all MCSs of w using suffix arrays (SA) and longest common prefix arrays (LCP) in $\mathcal{O}(n \log n)$ time.

Exact Number of MCSs in a Fibonacci Word: Given a Fibonacci word f_n, we give an exact equation to find the number of MCSs in f_n is denoted by $M(f_n)$ and is defined in Equation (8). We also show that $M(f_n) \approx 1.382F_n$.

In Sect. 2, we introduce the terminology and data structures used in the paper. In Sect. 3 we define a compact representation of all closed substrings of a given string $w[1..n]$. In Sect. 4, we present algorithms to efficiently compute all closed substrings in a compact representation, as well as all MCSs, in $\mathcal{O}(n \log n)$ time. In Sect. 5 we provide a formula to compute the exact number of MCSs in a Fibonacci word. Finally, we present experimental analysis in Sect. 6.

2 Preliminaries

An alphabet Σ, of size σ, is a set of symbols that is totally ordered. A ***string (word)***, written as $w[1..n]$, is an element of Σ^*, whose length is represented by

n. The ***empty string*** of length zero is denoted by ε. A ***substring (factor)*** $w[i..j]$ of w is a string, where $1 \leq i, j \leq n$, if $j > i$ then the substring is ε. The substring $w[i..j]$ is called a ***proper substring***, if $j - i + 1 < n$. A substring $w[i..j]$ where $i = 1$ ($j = n$) is called a ***prefix*** (***suffix***) of w. If a substring u of w is both a proper prefix and a proper suffix of w, then it is called a ***border***. A ***cover*** of a string w is a substring u of w such that w can be constructed by overlapping and/or adjacent instances of u. If $|u| < |w|$, then it is said to be a ***proper cover*** (for more details on covers, see the survey [19]).

A string $w[1..n]$ is ***periodic*** with period t' if $w[i] = w[i + t']$ holds for all $1 \leq i \leq n-t'$. The ***shortest period*** of w is the smallest positive integer $t \leq t'$ for which this condition holds. A periodic substring $w[i..j]$ of string $w[1..n]$ with shortest period t is called a ***run*** if its length $j - i + 1 \geq 2t$ and its periodicity cannot be extended to the left or right, that is, $i = 1$ or $w[i-1] \neq w[i+t-1]$, and $j = n$ or $w[j + 1] \neq w[j - t + 1]$.

A ***gapped repeat*** is a substring of $w[1..n]$ of the form uvu, where u and v are non-empty strings. The substring u is called the ***arm*** of the gapped repeat. The period[1] of the gapped repeat is $|u| + |v|$. An occurrence of a gapped repeat is called a ***maximal gapped repeat*** if it cannot be extended to the left or to the right by at least one symbol while preserving its period. A maximal gapped repeat, which is also closed, is referred to as a ***gapped-MCS***.

The ***suffix array*** $\mathsf{SA}[1..n]$ of $w[1..n]$ is an integer array of length n, where each entry $\mathsf{SA}[i]$ points to the starting position of the i-th lexicographically least suffix of w. The ***longest common prefix array*** $\mathsf{LCP}[1..n]$ of $w[1..n]$ is an array of integers that represent the length of the longest common prefix of suffixes $\mathsf{SA}[i-1]$ and $\mathsf{SA}[i]$ for all $2 \leq i \leq n$, we assume $\mathsf{LCP}[1] = 0$.

A closed substring $u = w[i..j]$ of a string $w[1..n]$ is said to be ***right-extendible*** to a maximal right-closed substring $r = w[i..j']$ if $j < j' \leq n$ and every prefix $w[i..k]$ of r with $j < k < j'$ is also closed. In this case, r is the unique maximal right-closed extension of u.

3 Compact Representation of All Closed Substrings

A string $w[1..n]$ contains $\mathcal{O}(n^2)$ closed substrings, which can naively be computed in $\mathcal{O}(n^2)$ time by computing the border of all substrings of w. In this section, we define a compact representation for all $\mathcal{O}(n^2)$ closed substrings in Theorem 1 that requires only $\mathcal{O}(n \log n)$ space. Algorithm 2 in Sect. 4 presents an $\mathcal{O}(n \log n)$ time solution to compute all closed substrings of w in a compact representation of size $\mathcal{O}(n \log n)$.

Let u and v be the proper closed prefixes of the maximal right-closed substring r. Then, by definition of right-extendibility we define an equivalence relation R as follows:

$$R = \{(u, v) \mid u \text{ and } v \text{ are both right-extendible to } r\}.$$

[1] Note that the term period, as used in the definition of a gapped repeat, differs from its earlier usage in the context of periodic strings. Throughout this paper, the term period is consistently used in reference to periodic strings or runs.

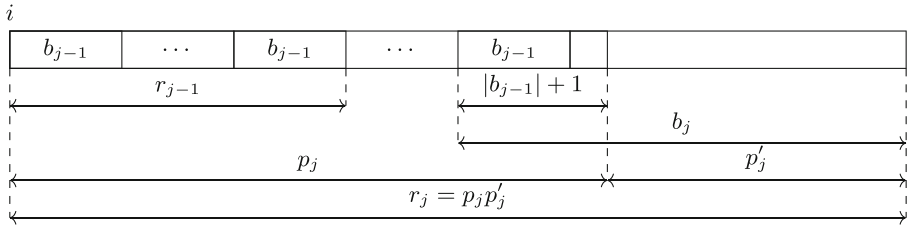

Fig. 1. Illustration of the second case in the proof of Lemma 2, showing that $|p_j|$ can be computed from $|b_j|$, $|b_{j-1}|$ and $|r_j|$, such that $p_j = r_j$ or $p_j \in E_{r_j}$. Note that when $p_j = r_j$, $p'_j = \varepsilon$.

Next, we define the equivalence class E_r of r under the relation R as follows:

$$E_r = \{u \mid u \text{ is right-extendible to } r\}. \tag{1}$$

Lemma 1. *Given a string $w[1..n]$, if $w[1..m]$ is the shortest maximal right-closed prefix of w, then $w[1..m] = \lambda^m$, where $\lambda \in \Sigma$ and $1 \leq m \leq n$. Moreover, each prefix of λ^m is also closed.*

Proof. We prove the lemma in two parts. First part by contradiction. Suppose that $w[1..m]$ is the shortest maximal right-closed prefix of w, and that $w[1..m] \neq \lambda^m$. W.l.o.g let $w[1] = \lambda$. Let $1 < j \leq m$ be the smallest index such that $w[j] = \lambda_1$ and $w[1..j-1] = \lambda^{j-1}$, where $\lambda_1 \neq \lambda \in \Sigma$. Then, $w[1..j] = \lambda^j \lambda_1$. However, by definition of maximal right-closed string, $w[1..j-1] = \lambda^{j-1}$ is a shorter maximal right-closed string of w—a contradiction.

Second, for any string of the form $u = \lambda^m$, consider its prefixes $p = u[1..j]$, where $1 \leq j \leq m$. If $j = 1$, then p is closed trivially. If $j \geq 2$, then the border length of each prefix p is equal to $j - 1$, which is maximum and unique. Thus, all the prefixes of $u = \lambda^m$ are closed. □

Let r denote a maximal right-closed substring, b its border, and p its shortest closed prefix such that $p = r$ or $p \in E_r$. Let $r_1, r_2, \ldots, r_{K_i}$ be all the maximal right-closed substrings starting at index i of a string $w[1..n]$, where $1 \leq \mathrm{i} \leq n$, and K_i is the number of maximal right-closed substrings starting at index i, such that $|r_1| < |r_2| < \cdots < |r_{K_i}|$.

In Eq. (2) we present a formula to compute the length of the shortest closed prefix p_j of r_j such that $p_j = r_j$ or $p_j \in E_{r_j}$, where $1 \leq j \leq K_i$. Lemma 2 proves Eq. (2) and is illustrated in Fig. 1.

Lemma 2. *Let $1 \leq j \leq K_i$. The length of the shortest closed prefix p_j of r_j, such that $p_j = r_j$ or $p_j \in E_{r_j}$, is given by:*

$$|p_j| = \begin{cases} 1, & \text{if } j = 1, \\ |r_j| - |b_j| + |b_{j-1}| + 1, & \text{if } 1 < j \leq K_i. \end{cases} \tag{2}$$

where b_j and b_{j-1} are the longest borders of r_j and r_{j-1}, respectively.

Proof. We prove the Lemma by considering two cases. For $j = 1$, by Equation (1) and Lemma 1, all prefixes of r_1 are closed and $|p_1| = 1$.

For $1 < j \leq K_i$, by definition of a closed substring, the shortest closed prefix p_j starting at index i and longer than r_{j-1} (whose longest border has length $|b_{j-1}|$) must have the longest border of length exactly $|b_{j-1}| + 1$. Either $p_j = r_j$ or by definition of right-extendibility p_j extends to $r_j = p_j p'_j$, with the longest border b_j, and each prefix of r_j with length at least $|p_j|$ is closed. Since $|p'_j| = |b_j| - (|b_{j-1}| + 1)$, we have $|r_j| = |p_j| + |p'_j| = |p_j| + |b_j| - (|b_{j-1}| + 1)$. Solving for $|p_j|$, we obtain the desired formula: $|p_j| = |r_j| - |b_j| + |b_{j-1}| + 1$. □

Lemma 3 (Theorem 1 in [18]). *A string $w[1..n]$ contains at most $\mathcal{O}(n \log n)$ maximal right (left)-closed substrings.*

Theorem 1 follows directly from Eq. (1), Lemma 2 and 3.

Theorem 1. *The compact representation of all closed substrings of $w[1..n]$ requires at most $\mathcal{O}(n \log n)$ space and is given by:*

$$\mathcal{C}(w) = \{(i, |p_j|, |r_j|) \mid 1 \leq i \leq n,\ 1 \leq j \leq K_i\}, \tag{3}$$

where K_i is the number of maximal right-closed substrings starting at position i, and for each j-th such substring r_j, starting at index i, p_j is the shortest closed prefix of r_j such that $p_j = r_j$ or $p_j \in E_{r_j}$.

The set of all closed substrings of $w[1..n]$, denoted by Closed(w), can be enumerated using the following equation:

$$\text{Closed}(w) = \bigcup_{(i,|p_j|,|r_j|) \in \mathcal{C}(w)} \{w[i \ldots i + \ell - 1] \mid \ell \in [|p_j|, |r_j|]\}. \tag{4}$$

An example of $\mathcal{C}(w)$ and the enumeration Closed(w) for the string $w[1..11] =$ *mississippi* is given in Table 2 in the Appendix.

4 Algorithms for Closed Substrings and MCSs

In this section, we introduce the primary data structure –the $\mathcal{MRC}$ array– which is used to compute all closed substrings in a compact representation and MCSs in $\mathcal{O}(n \log n)$ time. See Table 1 for an example of the $\mathcal{MRC}$ array for the string $w[1..11] =$ *mississippi* in the Appendix.

Definition 1 ($\mathcal{MRC}$ Array[2]). *Given a string $w[1..n]$, the **maximal right-closed array** ($\mathcal{MRC}$) is an array of size n, where $\mathcal{MRC}[i]$, $1 \leq i \leq n$, stores the list of ordered pairs $(|r_j|, |b_j|)$, where $j = K_i$ to 1.*

Algorithm 1 computes the $\mathcal{MRC}$ array by first identifying all the maximal right-closed substrings of length greater than one. It uses a stack to store AVL trees created during its execution. The algorithm scans the LCP array from left to

[2] This definition uses the notation introduced in Sect. 3.

right. Beginning with $\mathsf{LCP}[1] = 0$, and for increasing $\mathsf{LCP}[i]$ values, the algorithm creates a single node (root) AVL tree with $key = \mathsf{SA}[i]$ and $value = \mathsf{LCP}[i]$ and pushes it onto the stack, until either the LCP value decreases or the end of the array is reached.

We denote by LCP_{max} the value stored in the root node of the AVL tree on the top of the stack. When a decrease in the LCP value (or the end of the array) is encountered, the algorithm pops a collection of AVL trees from the stack, denoted as $\mathcal{L}_{suffixSets}$, such that all suffixes represented by an AVL tree key share the same longest common prefix equal to LCP_{max}. Since LCP_{max} is the maximum LCP value of all the AVL trees on the stack, popping this collection, $\mathcal{L}_{suffixSets}$, preserves the strictly decreasing order of ordered pairs in the $\mathcal{MRC}$ array. This collection is then merged into a single AVL tree using the *Union* operation presented in [16][3], resulting in $\mathcal{T}_{suffixes} = \bigcup \mathcal{L}_{suffixSets}$. The *Union* operation also computes a list of ordered pairs representing maximal right-closed substrings via the $ChangeList(\mathcal{L}_{suffixSets}, \mathcal{T}_{suffixes})$ operation defined as follows:

$$ChangeList(\mathcal{P}, \mathcal{P}') = \{(x, next_{\mathcal{P}'}(x)) : next_{\mathcal{P}}(x) \neq next_{\mathcal{P}'}(x)\}, \tag{5}$$

where $\mathcal{P}$ is a partition of the set $\mathcal{P}'$, and $next_X(x) = \min\{y \in X \mid y > x\}$, $x \in X$ and $X \subseteq \mathbb{Z}^+$. In particular, if $x \in P_i$ for some $P_i \in \mathcal{P}$, then $next_{\mathcal{P}}(x) = next_{P_i}(x)$. Each pair $(x, y) \in ChangeList(\mathcal{L}_{suffixSets}, \mathcal{T}_{suffixes})$ identifies a maximal right-closed substring $r_j = w[x..y + LCP_{max} - 1]$, where LCP_{max} is the length of its longest border. Hence, the pair $(y + LCP_{max} - x, LCP_{max})$ is added to $\mathcal{MRC}[x]$. All such pairs from the $ChangeList$ are added to the $\mathcal{MRC}$ array. The algorithm then assigns to the root node of the AVL tree, $\mathcal{T}_{suffixes}$, the LCP value of the AVL tree at the top of the stack and pushes it onto the stack, only if the stack is not empty–thus maintaining the lexicographic ordering of the set of suffixes–and continues scanning the LCP array.

To compute the maximal right-closed substrings of length one, it performs a simple linear scan of the string w and adds $w[i]$ to $\mathcal{MRC}[i]$, if $i = n \vee (1 \leq i < n \wedge w[i] \neq w[i+1])$.

It is known that LCP array identifies all the occurrences of the repeating substrings that cannot be extended to the right. Moreover, the suffix array groups all these occurrences within specific ranges. Algorithm 1 identifies such ranges by tracking the rise and fall of values in the LCP array, and stores the corresponding indices in $\mathcal{L}_{suffixSets}$ as keys in AVL trees, starting with the substring of length LCP_{max}. The *Union* operation then merges these single node AVL trees to form one AVL tree ($\mathcal{T}_{suffixes}$) that contains all the SA values; that is, all the starting positions of the substrings of length LCP_{max} in the string w.

The $ChangeList(\mathcal{L}_{suffixSets}, \mathcal{T}_{suffixes})$ operation then returns every pair of consecutive occurrences of substrings in the $\mathcal{L}_{suffixSets}$. These consecutive occurrences form the border of the maximal right-closed substring and are therefore added to the $\mathcal{MRC}$ array. Finally, the simple linear scan adds maximal

[3] The *Union* operation, defined in [16], builds upon foundational methods from [8,9].

Algorithm 1. Compute $\mathcal{MRC}$ Array

Input: A string $w[1..n]$, its $\mathsf{SA}[1..n]$ and $\mathsf{LCP}[1..n]$.
Output: The $\mathcal{MRC}$ array of string $w[1..n]$.

```
1: i ← 0, MRC ← {∅}^n, stack ← ∅
2: while stack ≠ ∅ or i < n do
3:     while i < n and (stack = ∅ or LCP[i] ≥ LCP(top(stack))) do
4:         push(stack, AVLTree({SA[i]}, LCP[i]))
5:         i ← i + 1
6:     L_suffixSets ← ∅
7:     LCP_max ← LCP(top(stack))
8:     while stack ≠ ∅ and LCP(top(stack)) = LCP_max do
9:         insert(L_suffixSets, pop(stack))
10:    if LCP_max ≠ 0 then
11:        previousLCP ← LCP(top(stack))
12:        insert(L_suffixSets, pop(stack))
13:        T_suffixes = ⋃ L_suffixSets          ▷ ⋃ L_suffixSets returns an AVL tree
14:        foreach (x, y) ∈ ChangeList(L_suffixSets, T_suffixes) do      ▷ Equation (5)
15:            insert(MRC[x], (y + LCP_max − x, LCP_max))
16:        LCP(T_suffixes) ← previousLCP
17:        push(stack, T_suffixes)
18: for i ← 1 to n do
19:    if i = n or (i < n and w[i] ≠ w[i + 1]) then
20:        insert(MRC[i], (1, 0))
21: return MRC
```

right-closed substrings of length one. The algorithm clearly computes the maximal right-closed substrings in strictly decreasing order of length as required in the $\mathcal{MRC}$ array. Thus, the correctness of Algorithm 1 follows.

We note that the algorithm in the worst case performs $4n-2$ stack operations for a string $w = \lambda^n$ and in the best case performs $2n$ stack operations for a string with all distinct characters.

Lemma 4 ([16]). *The Union operation correctly computes the ChangeList, and any sequence of Union operations takes $\mathcal{O}(n \log n)$ time in total.*

Theorem 2. *Algorithm 1 correctly computes the $\mathcal{MRC}$ array of a string $w[1..n]$ using SA and LCP in $\mathcal{O}(n \log n)$ time.*

Proof. We prove the total time complexity of Algorithm 1 in three parts. First, for stack operations, we introduce a potential function $\Phi(S) = |S| \geq 0$, where $|S|$ is the number of elements in the stack. The actual cost of each push and pop operation is 1. Since we push all n single node AVL trees corresponding to each suffix onto the stack, the amortized cost for each push operation is $c'_{push} = 1 + \Delta\Phi(S) = 2$, resulting in a total amortized cost of $2n$. Additionally, in the algorithm, we pop at least two AVL trees and push their *Union* back onto the stack, and because the amortized cost of this sequence of pops and push in

Algorithm 2. Compute $\mathcal{C}(w)$

Input: $\mathcal{MRC}$ array of strings of $w[1..n]$
Output: Compact representation $\mathcal{C}(w)$ as in Eq. (3)

```
1: C ← ∅
2: for i ← 1 to n do
3:     for j ← 1 to length of MRC[i] do
4:         (lr_j, lb_j) ← MRC[i][j]                    ▷ lr_j = |r_j|, lb_j = |b_j|, lp_j = |p_j|
5:         if j = 1 then
6:             lp_j ← 1                                 ▷ Eq. (2)
7:         else
8:             (lr_{j-1}, lb_{j-1}) ← MRC[i][j − 1]
9:             lp_j ← lr_j − lb_j + lb_{j-1} + 1        ▷ Eq. (2)
10:        insert(C, (i, lp_j, lr_j))
11: return C
```

the worse case is $c'_{pops-push} = 3 + \Delta\Phi(S) = 2$. The maximum number of such operations is $n - 1$, leading to a total amortized cost of $2(n - 1)$. Therefore, the total cost of all stack operations in the worst case is $4n - 2$, which is $\mathcal{O}(n)$. Second, by Lemma 4, the time complexity of the sequence of *Union* operations is $\mathcal{O}(n \log n)$. Finally, the single-characters that are maximal right-closed are computed by a linear scan on w, taking $\mathcal{O}(n)$ time. Therefore, the total time complexity is $\mathcal{O}(n \log n)$. □

Algorithm to Compute $\mathcal{C}(w)$: Algorithm 2 computes the compact representation $\mathcal{C}(w)$, as in Equation (2), of all closed substrings of a string $w[1..n]$, by simply traversing all n lists in the $\mathcal{MRC}$ array (see example in Table 2 in the Appendix). Since the $\mathcal{MRC}$ array is of size $\mathcal{O}(n \log n)$ we get Theorem 3.

Theorem 3. *Algorithm 2 correctly computes all* $\mathcal{O}(n^2)$ *closed substrings of a string* $w[1..n]$ *in a compact representation in* $\mathcal{O}(n \log n)$ *time.*

Algorithm to Compute All MCSs: Algorithm 3 computes all MCSs of a string $w[1..n]$ by iterating through the $\mathcal{MRC}$ array (see example in Table 2 in the Appendix). For each index i in w (where $1 \leq i \leq n$), the algorithm examines each maximal right-closed substring represented in $\mathcal{MRC}[i]$ and verifies whether the substring is also maximal left-closed using a constant time check $i = 1$ or ($i >$ 1 and $w[i-1] \neq w[i + |r| - |b| - 1]$). If the condition is satisfied, the substring is added to the MCS list. Since scanning through the $\mathcal{MRC}$ array takes $\mathcal{O}(n \log n)$ time, and each check is performed in constant time, the algorithm computes all MCSs in $\mathcal{O}(n \log n)$ time, establishing Theorem 4.

Theorem 4. *Algorithm 3 correctly computes all MCSs of a string* $w[1..n]$ *in* $\mathcal{O}(n \log n)$ *time.*

5 MCSs in a Fibonacci Word

In this section, we study the occurrences of MCSs in a Fibonacci word (f_n), and provide a formula to compute the exact number of MCSs in f_n.

Algorithm 3. Compute all MCSs

Input: A string $w[1..n]$ and $\mathcal{MRC}$ array
Output: All MCSs

```
1: mcsList ← ∅
2: for i ← 1 to n do
3:     for j ← 1 to length of MRC[i] do
4:         (lr, lb) ← MRC[i][j]                          ▷ lr = |r|, lb = |b|
5:         if i = 1 or (i > 1 and w[i − 1] ≠ w[i + lr − lb − 1]) then
6:             insert(mcsList, w[i..i + lr − 1])
7: return mcsList
```

Let us recall that a ***Fibonacci*** word f_n is defined recursively by $f_0 = 0$, $f_1 = 1$, and $f_n = f_{n-1}f_{n-2}$ for $n \geq 2$. We denote $F_n = |f_n|$, where F_n satisfies the Fibonacci recurrence relation $F_n = F_{n-1} + F_{n-2}$, with $F_0 = 1$ and $F_1 = 1$. We refer to the logical separation between f_{n-1} and f_{n-2}, in the string f_n, as the ***boundary***.

Bucci et al. [10] study the open and closed prefixes of Fibonacci words and show that the sequence of open and closed prefixes of a Fibonacci word follows the Fibonacci sequence. De Luca et al. [11] explore the sequence of open and closed prefixes of a Sturmian word. In [14], the authors define and find the closed Ziv–Lempel factorization and classify closed prefixes of infinite m-bonacci words. In this section, we extend previous work on closed substrings in Fibonacci words by deriving a formula to compute the number of MCSs they contain.

For any integer $n \geq 2$, let $p_n = f_n[1..F_n - 2]$ denote the prefix of f_n excluding its last two symbols. Then, $f_n = p_n\,\delta_n$, where $\delta_n = 10$ if n is even and $\delta_n = 01$, otherwise. Let $P_n = F_n - 2$ denote the length of p_n. The golden ratio is defined as $\phi = \frac{1+\sqrt{5}}{2}$, and it is known that the ratio of consecutive Fibonacci numbers converges to ϕ, i.e., $\lim_{n\to\infty} \frac{F_n}{F_{n-1}} = \phi$.

The non-singleton MCSs are either runs or maximal gapped repeats in which the arm occurs exactly twice [4]. For a Fibonacci word f_n, let $SM(f_n)$, $R(f_n)$, and $GM(f_n)$ denote the number of singleton MCSs, runs, and gapped-MCSs, respectively. Therefore, the total number of MCSs in f_n is given by $M(f_n) = SM(f_n) + R(f_n) + GM(f_n)$.

Lemma 5. *The no. of singleton MCSs in a Fibonacci word f_n, for $n \geq 4$, is:*

$$SM(f_n) = \begin{cases} F_{n-2} + F_{n-4} + 2, & \text{if } n \text{ is odd,} \\ F_{n-2} + F_{n-4}, & \text{if } n \text{ is even.} \end{cases} \tag{6}$$

Proof. We prove the Lemma by induction on n. The base cases, $SM(f_4) = 3$ and $SM(f_5) = 6$, match the direct counts.

For the inductive step, assume that Eq. (6) holds for $k \geq 6$. By definition, $f_{k+1} = f_k f_{k-1}$. Every Fibonacci word f_k begins with a 10; if k is odd, it ends with 01 and if k is even, it ends with 10. By definition, the first and last character in f_k are singleton MCSs. Therefore, we examine whether the singleton MCSs at the boundary are retained in f_{k+1}.

Suppose $k+1$ is even. By inductive hypothesis, $SM(f_k) = F_{k-2} + F_{k-4} + 2$ and $SM(f_{k-1}) = F_{k-3} + F_{k-5}$. Since k is odd, f_k ends with 1 and f_{k-1} begins with 1, their concatenation results in the string 11 at the boundary, which forms a new non-singleton MCS, and reduces the count of the singleton MCSs in f_{k+1} by two. Thus, we get $SM(f_{k+1}) = SM(f_k) + SM(f_{k-1}) - 2 = F_{k-1} + F_{k-3}$.

Suppose $k+1$ is odd. By inductive hypothesis, $SM(f_k) = F_{k-2} + F_{k-4}$ and $SM(f_{k-1}) = F_{k-3} + F_{k-5} + 2$. Since k is even, f_k ends with 0 and f_{k-1} begins with 1, their concatenation results in the string 01 at the boundary, which does not reduce the count of singleton MCSs in f_{k+1}. Thus, we get $SM(f_{k+1}) = SM(f_k) + SM(f_{k-1}) = F_{k-1} + F_{k-3} + 2$. □

Lemma 6 (Theorem 2 in [17]). *For $n \geq 4$, the number of runs in the Fibonacci word f_n, is given by $R(f_n) = 2F_{n-2} - 3$.*

To compute $GM(f_n)$, we count the number of maximal gapped repeats whose arms occur exactly twice, as these are, by definition gapped-MCSs. In Theorem 5 we first show that the only gapped-MCSs in a Fibonacci word are occurrences of the substring 101 that are also maximal gapped repeats. Then, we count all such occurrences in f_n.

Lemma 7 (Theorem 1 in [27]). *Suppose uvu is a maximal gapped repeat in f_n that is also a suffix of f_n. Then, the arm u is a Fibonacci word.*

Lemma 8. *For $n \geq 5$, suppose uvu is a maximal gapped repeat in f_n that is also a suffix of f_n. Then uvu is not a gapped-MCS.*

Proof. By Lemma 7, the arm of the maximal gapped repeat is a Fibonacci word $u = f_k$. Clearly, $k \neq n$.

If n is even, then the proper suffixes of f_n that are Fibonacci words are in the set $\mathcal{S} = \{f_k \mid k \mod 2 = 0 \wedge 0 \leq k < n\}$. For any proper suffix $f_k \in \mathcal{S} \setminus \{f_0, f_2\}$, f_{k+2} is a suffix of f_n. By definition, $f_{k+2} = f_{k+1}f_k = f_kf_{k-1}f_k = f_kf_kf_{k-3}f_{k-2}$, the suffix $f_{k-1}f_k = f_kf_{k-3}f_{k-2}$ has length $F_k + F_{k-1} < 2F_k$ implying that the last two occurrences of f_k overlap. Therefore, any uvu with f_k as its arm, will have at least three occurrences of f_k making it an open string. For suffix $f_k \in \{f_0, f_2\}$ of f_n, for $n \geq 6$, $f_6 = 1011010110110$ is a suffix of f_n, and the last two occurrences of f_k are left-extendible, and so, it is not maximal. Moreover, any longer uvu with the arm f_k will have at least three occurrences of f_k making it an open string.

If n is odd, then $\mathcal{S} = \{f_k \mid k \mod 2 = 1 \wedge 1 \leq k < n\}$. The argument is analogous for $f_k \in \mathcal{S} \setminus \{f_1\}$ and $f_k \in \{f_1\}$, respectively. □

Lemma 9 (Theorem 2 in [27]). *For $n \geq 3$, suppose uvu is a maximal gapped repeat in f_n that is not a suffix of f_n. Then the arm u belongs to the set $\mathcal{A}_n = \{p_3, p_4, \ldots, p_{n-2}\}$.*

Lemma 10 (Lemma 8 in [27]). *For $n \geq 4$, all the borders of p_n form the set $\mathcal{B}_n = \{p_3, p_4, \ldots, p_{n-1}\}$.*

Lemma 11. *For $n \geq 5$, the border p_{n-1} of p_n is the longest proper cover of p_n.*

Proof. By Lemma 10, we know that p_{n-1} is the longest border of p_n. $F_n = F_{n-1}+F_{n-2}$. For $n \geq 5$, $F_n-4 = F_{n-1}+F_{n-2}-4$, and so $P_n-2 = P_{n-1}+P_{n-2}$. Hence, $P_n \leq 2 \cdot P_{n-1}$. Clearly, p_{n-1} is either an adjacent or an overlapping border of p_n. Therefore, p_{n-1} is the longest proper cover of p_n. □

Lemma 12 (Lemma 2 in [21]). *Let u be a proper cover of x and let $v \neq u$ be a substring of x such that $|v| \leq |u|$. Then, v is a cover of $x \Leftrightarrow v$ is a cover of u.*

Lemma 13. *For $n \geq 5$, all covers of p_n form the set $\mathcal{C}_n = \{p_4, p_5, \ldots, p_n\}$.*

Proof. By Lemma 11, for $n \geq 5$, p_{n-1} is the longest proper cover of p_n. Similarly, p_{n-2} is the longest proper cover of p_{n-1}, p_{n-3} is the longest proper cover of p_{n-2}, and so on. Therefore, by Lemma 12, it follows that all covers of p_n belong to the set $\mathcal{C}_n = \{p_4, p_5, \ldots, p_n\}$ for all $n \geq 5$. □

Theorem 5. *For $n \geq 5$, every gapped-MCS in a Fibonacci word f_n corresponds to an occurrence of the substring* 101 *that is also a maximal gapped repeat.*

Proof. All gapped-MCSs are maximal gapped repeats. By Lemma 8, for $n \geq 5$, all maximal gapped repeats that are also suffixes of f_n are not gapped-MCSs.

By Lemma 9, for $n \geq 3$, if uvu is a maximal gapped repeat in f_n that is not a suffix of f_n, then the arm u belongs to the set $\mathcal{A}_n = \{p_3, p_4, \ldots, p_{n-2}\}$. By Lemma 13, all covers of p_n belong to the set $\mathcal{C}_n = \{p_4, p_5, \ldots, p_n\}$.

Suppose there exists a maximal gapped $w = uvu$, where $u \in \mathcal{A}_n \setminus p_3$, that is a gapped-MCS. Assume n is odd. Then, $f_n = p_n\delta_n$, where $\delta_n = 01$, and $f_n[F_n - 1] = 0$. Since $u \in \mathcal{A}_n \setminus p_3$, it ends with 1. Clearly, u does not occur ending at position $F_n - 1$. Now suppose n is even. By a similar reasoning, u does not occur ending at $F_n - 1$ since it ends with 01 and $f_n[F_n - 2] = f_n[F_n - 1] = 1$ (as $f_4 = 10110$ is a suffix of f_n). Hence, u ends within p_n. Since u is a cover of p_n, $w = uvu$ will contain at least three occurrences of u making it an open string. Therefore, there is no maximal gapped repeat of the form $w = uvu$, where $u \in \mathcal{A}_n \setminus p_3$, that is a gapped MCS.

Now we consider the final case, where $u = p_3 = 1$. In this case, we get 101 as the only gapped-MCS. Any longer maximal gapped repeat will have at least three occurrences of $u = p_3 = 1$, making it an open string—a contradiction. □

Lemma 14. *The no. of gapped-MCSs in a Fibonacci word f_n, for $n \geq 5$, is:*

$$GM(f_n) = \begin{cases} F_{n-5}, & \text{if } n \text{ is odd,} \\ F_{n-5} + 1, & \text{if } n \text{ is even.} \end{cases} \tag{7}$$

Proof. We prove the Lemma by induction on n. By direct counting for the base cases, we verify that $GM(f_5) = 1$ and $GM(f_6) = 2$. For the inductive step, assume that Eq. (7) holds for $k \geq 7$. For $n \geq 3$, $f_3 = 101$ is a prefix of every f_n, and by Theorem 5 every gapped-MCS in a Fibonacci word f_n, for $n \geq 5$,

corresponds to an occurrence of the substring 101 that is also a maximal gapped repeat. In $f_{k+1} = f_k f_{k-1}$, any new occurrences of 101 in f_{k+1} can only occur beginning in f_k and ending in f_{k-1}. Moreover, an occurrence of 101 as a suffix of f_k or a prefix of f_{k-1} may be lost by extension. Below we examine all such cases.

Suppose $k+1$ is odd. By inductive hypothesis, $GM(f_k) = F_{k-5} + 1$ and $GM(f_{k-1}) = F_{k-6}$. Moreover, f_k ends with the suffix $f_2 = 10$ and f_{k-1} begins with $f_3 = 101$, resulting in the formation of the string 101 crossing the boundary. Clearly, this string is not a gapped-MCS as it is right-extendible. The gapped-MCS 101 which is the prefix of f_{k-1} is no longer a gapped-MCS since f_k ends with a 0 making it left-extendible. Thus, we get $GM(f_{k+1}) = GM(f_k) + GM(f_{k-1}) - 1 = (F_{k-5} + 1) + F_{k-6} - 1 = F_{k-4}$.

Suppose $k+1$ is even. By inductive hypothesis, $GM(f_k) = F_{k-5}$ and $GM(f_{k-1}) = F_{k-6} + 1$. f_k ends with a 1, so the prefix 101 of f_{k-1} is retained as a gapped-MCS. By Lemma 8 the suffix $f_3 = 101$ of f_k is not a gapped-MCS. Thus, we get $GM(f_{k+1}) = GM(f_k) + GM(f_{k-1}) = F_{k-4} + 1$. □

Theorem 6. *The number of MCSs in a Fibonacci word f_n, denoted by $M(f_n)$, for $n \geq 5$, is given below in Eq. (8). Furthermore, $M(f_n) \approx 1.382 F_n$.*

$$M(f_n) = \begin{cases} F_n + F_{n-2} - 1, & \text{if } n \text{ is odd,} \\ F_n + F_{n-2} - 2, & \text{if } n \text{ is even.} \end{cases} \tag{8}$$

Proof. Adding Eqs. (6), (7) and the Eq. in Lemma 6 and simplifying, we get Eq. (8). Next, we have $M(f_n) < F_n + F_{n-2} = \left(1 + \frac{F_{n-2}}{F_n}\right) F_n \approx \left(1 + \frac{1}{\phi^2}\right) F_n \approx 1.382 F_n$ (since $\lim_{n\to\infty} \frac{F_{n-2}}{F_n} = \frac{1}{\phi^2}$). □

6 Experimental Analysis

All experiments were carried out on a server with an Intel Xeon Gold 6426Y CPU (64 cores, 128 threads, 75 MiB L3 cache), 250 GiB RAM, running RHEL 9.5 (kernel version `5.14.0-503.26.1`). The code was implemented in C++ and compiled using GCC 11.5.0. The implementations for computing the suffix array (SA) and the longest common prefix array (LCP) were sourced from the *libsais* library [13], which is based on contributions from [15,22,25,26]. The implementation is available at https://github.com/neerjamhaskar/closedStrings.

For $\sigma = 2, 3$ and 4, we generate all strings of length n for $1 \leq n \leq 20$ and compute their MCSs. The results are plotted in Fig. 2 in the Appendix. As observed, the maximum number of MCSs increases with the size of the alphabet σ for strings of the same length. In total, the number of strings analyzed is equal to $\sum_{i=1}^{20}(2^i + 3^i + 4^i) \approx 1.47$ trillion. These exhaustive computations raise a natural question: Given n and σ, can we algorithmically construct a string that has the maximum possible number of MCSs?

Acknowledgments. We thank Simon J. Puglisi for introducing us to the MCS problem using SA and LCP, which led to Theorem 4. We are grateful to the reviewers for their valuable feedback.

Appendix

Table 1. The $\mathcal{MRC}$ array for the string $w[1..11] = mississippi$.

Index (i)	$\mathcal{MRC}[i]$
1	$(1, 0)$
2	$(7, 4), (1, 0)$
3	$(6, 3), (2, 1)$
4	$(5, 2), (3, 1), (1, 0)$
5	$(4, 1), (1, 0)$
6	$(2, 1)$
7	$(1, 0)$
8	$(4, 1), (1, 0)$
9	$(2, 1)$
10	$(1, 0)$
11	$(1, 0)$

Table 2. Compact representation $\mathcal{C}(w)$ of $w[1..11] = mississippi$ with corresponding closed factors, maximal right-closed factors, and MCSs.

$C(w)$	All Closed Factors	Maximal Right Closed Factors	MCS?
$(1, 1, 1)$	m	m	Yes
$(2, 4, 7)$	$issi$, $issis$, $ississ$, $ississi$	$ississi$	Yes
$(2, 1, 1)$	i	i	Yes
$(3, 5, 6)$	$ssiss$, $ssissi$	$ssissi$	No
$(3, 1, 2)$	s, ss	ss	Yes
$(4, 5, 5)$	$sissi$	$sissi$	No
$(4, 3, 3)$	sis	sis	Yes
$(4, 1, 1)$	s	s	No
$(5, 4, 4)$	$issi$	$issi$	No
$(5, 1, 1)$	i	i	Yes
$(6, 1, 2)$	s, ss	ss	Yes
$(7, 1, 1)$	s	s	No
$(8, 4, 4)$	$ippi$	$ippi$	Yes
$(8, 1, 1)$	i	i	Yes
$(9, 1, 2)$	p, pp	pp	Yes
$(10, 1, 1)$	p	p	No
$(11, 1, 1)$	i	i	Yes

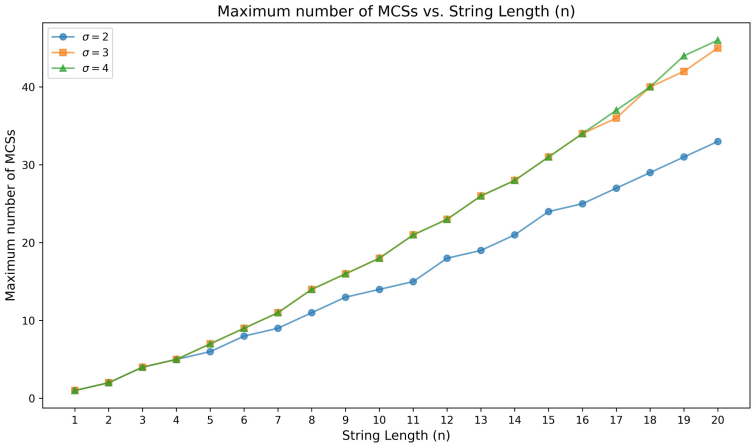

Fig. 2. Maximum number of MCSs in strings of length n, where $1 \leq n \leq 20$, for alphabets of size $\sigma = 2, 3$ and 4.

References

1. Alamro, H., Alzamel, M., Iliopoulos, C.S., Pissis, S.P., Watts, S., Sung, W.-K.: Efficient identification of k-closed strings. In: Boracchi, G., Iliadis, L., Jayne, C., Likas, A. (eds.) EANN 2017. CCIS, vol. 744, pp. 583–595. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-65172-9_49
2. Badkobeh, G., et al.: Closed factorization. In: Holub, J., Zdařek, J. (eds.) Proceedings of PSC 2014, pp. 162–168. Czech Technical University in Prague (2014). https://www.stringology.org/papers/PSC2014.pdf
3. Badkobeh, G., et al.: Closed factorization. Disc. Appl. Math. **212**, 23–29 (2016). https://doi.org/10.1016/j.dam.2016.04.009
4. Badkobeh, G., De Luca, A., Fici, G., Puglisi, S.J.: Maximal closed substrings. In: Arroyuelo, D., Poblete, B. (eds.) String Processing and Information Retrieval, pp. 16–23. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-20643-6_2
5. Badkobeh, G., Fici, G., Lipták, Z.: On the number of closed factors in a word. In: Dediu, A.-H., Formenti, E., Martín-Vide, C., Truthe, B. (eds.) LATA 2015. LNCS, vol. 8977, pp. 381–390. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-15579-1_29
6. Badkobeh, G., Luca, A.D., Fici, G., Puglisi, S.: Maximal closed substrings (2022). https://doi.org/10.48550/arXiv.2209.00271
7. Bannai, H., et al.: Efficient algorithms for longest closed factor array. In: Iliopoulos, C., Puglisi, S., Yilmaz, E. (eds.) SPIRE 2015. LNCS, vol. 9309, pp. 95–102. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-23826-5_10
8. Brodal, G.S., Pedersen, C.N.S.: Finding maximal quasiperiodicities in strings. In: Giancarlo, R., Sankoff, D. (eds.) CPM 2000. LNCS, vol. 1848, pp. 397–411. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-45123-4_33

9. Brown, M.R., Tarjan, R.E.: A fast merging algorithm. J. ACM **26**(2), 211–226 (1979). https://doi.org/10.1145/322123.322127
10. Bucci, M., De Luca, A., Fici, G.: Enumeration and structure of Trapezoidal words. Theor. Comput. Sci. **468**, 12–22 (2013). https://doi.org/10.1016/j.tcs.2012.11.007
11. De Luca, A., Fici, G., Zamboni, L.Q.: The sequence of open and closed prefixes of a Sturmian word. Adv. Appl. Math. **90**, 27–45 (2017). https://doi.org/10.1016/j.aam.2017.04.007
12. Fici, G.: A classification of Trapezoidal words. In: Ambrož, P., Štěpán Holub, Masáková, Z. (eds.) Proceedings 8th International Conference Words 2011, Prague, Czech Republic, 12–16 September 2011. Electronic Proceedings in Theoretical Computer Science, vol. 63, pp. 129–137. Open Publishing Association (2011). https://doi.org/10.4204/EPTCS.63.18
13. Grebnov, I.: libsais: a library for linear time suffix array, longest common prefix array and burrows wheeler transform construction based on induced sorting algorithm. GitHub repository (2021–2025). https://github.com/IlyaGrebnov/libsais, version 2.10.2. Accessed 12 June 2025
14. Jahannia, M., Mohammad-Noori, M., Rampersad, N., Stipulanti, M.: Closed Ziv–Lempel factorization of the m-bonacci words. Theor. Comput. Sci. **918**, 32–47 (2022). https://doi.org/10.1016/j.tcs.2022.03.019
15. Kärkkäinen, J., Manzini, G., Puglisi, S.J.: Permuted longest-common-prefix array. In: Kucherov, G., Ukkonen, E. (eds.) CPM 2009. LNCS, vol. 5577, pp. 181–192. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-02441-2_17
16. Kociumaka, T., Pissis, S.P., Radoszewski, J., Rytter, W., Waleń, T.: Fast algorithm for partial covers in words. Algorithmica **73**(1), 217–233 (2015). https://doi.org/10.1007/s00453-014-9915-3
17. Kolpakov, R., Kucherov, G.: On maximal repetitions in words. In: Ciobanu, G., Păun, G. (eds.) FCT 1999. LNCS, vol. 1684, pp. 374–385. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48321-7_31
18. Kosolobov, D.: Closed repeats (2024). https://doi.org/10.48550/arXiv.2410.00209
19. Mhaskar, N., Smyth, W.: String covering: a survey. Fund. Inform. **190**(1), 17–45 (2023). https://doi.org/10.3233/FI-222164
20. Mieno, T., Takahashi, S., Seto, K., Horiyama, T.: Online and offline algorithms for counting distinct closed factors via sliding suffix trees. In: Královič, R., Kůrková, V. (eds.) SOFSEM 2025: Theory and Practice of Computer Science, pp. 172–183. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-82697-9_13
21. Moore, D., Smyth, W.: An optimal algorithm to compute all the covers of a string. Inf. Process. Lett. **50**(5), 239–246 (1994). https://doi.org/10.1016/0020-0190(94)00045-X
22. Nong, G., Zhang, S., Chan, W.H.: Two efficient algorithms for linear time suffix array construction. IEEE Trans. Comput. **60**(10), 1471–1484 (2011). https://doi.org/10.1109/TC.2010.188
23. Parshina, O., Puzynina, S.: Finite and infinite closed-rich words. Theor. Comput. Sci. **984**, 114315 (2024). https://doi.org/10.1016/j.tcs.2023.114315
24. Parshina, O., Zamboni, L.Q.: Open and closed factors in Arnoux-Rauzy words. Adv. Appl. Math. **107**, 22–31 (2019). https://doi.org/10.1016/j.aam.2019.02.007
25. Timoshevskaya, N., Feng, W.c.: Sais-opt: on the characterization and optimization of the sa-is algorithm for suffix array construction. In: 2014 IEEE 4th International Conference on Computational Advances in Bio and Medical Sciences (ICCABS), pp. 1–6 (2014). https://doi.org/10.1109/ICCABS.2014.6863917

26. Xie, J.Y., Nong, G., Lao, B., Xu, W.: Scalable suffix sorting on a multicore machine. IEEE Trans. Comput. **69**(9), 1364–1375 (2020). https://doi.org/10.1109/TC.2020.2972546
27. Yamane, K., Nakashima, Y., Seto, K., Horiyama, T.: Maximal α-gapped repeats in a Fibonacci string. In: Královič, R., Kůrková, V. (eds.) SOFSEM 2025: Theory and Practice of Computer Science, pp. 337–350. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-82697-9_25